



EAST WEST UNIVERSITY

Lab 7

Implementation of Circle Drawing Algorithms

Course Title: Computer Graphics

Course Code: CSE 420

Semester: Spring 2026

Section: 01

Submitted by

Name: Shairin Akter Hashi

ID: 2022-2-60-102

Date: 09/03/2026

Submitted to

Dr. Md. Tauhid Bin Iqbal

Assistant Professor

Department of Computer Science and Engineering

East West University

Lab Task

1. Midpoint Circle Drawing Algorithm

```
#include<stdio.h>
#include <GL/gl.h>
#include <GL/glut.h>
#include <math.h>

// Global variables
int xc, yc, r;

// Function to plot the 8 symmetric points
void plotCirclePoints(int xc, int yc, int x, int y)
{
    glVertex2f((xc + x)/100.0, (yc + y)/100.0);
    glVertex2f((xc - x)/100.0, (yc + y)/100.0);
    glVertex2f((xc + x)/100.0, (yc - y)/100.0);
    glVertex2f((xc - x)/100.0, (yc - y)/100.0);

    glVertex2f((xc + y)/100.0, (yc + x)/100.0);
    glVertex2f((xc - y)/100.0, (yc + x)/100.0);
    glVertex2f((xc + y)/100.0, (yc - x)/100.0);
    glVertex2f((xc - y)/100.0, (yc - x)/100.0);

    // Print points in console
    printf("(d,d)\n", xc+x, yc+y);
    printf("(d,d)\n", xc-x, yc+y);
    printf("(d,d)\n", xc+x, yc-y);
    printf("(d,d)\n", xc-x, yc-y);
    printf("(d,d)\n", xc+y, yc+x);
    printf("(d,d)\n", xc-y, yc+x);
    printf("(d,d)\n", xc+y, yc-x);
    printf("(d,d)\n", xc-y, yc-x);
}

// Display function
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0,1.0,1.0);
    glPointSize(5.0);

    int x = 0;
    int y = r;

    // Initial decision parameter
```

```

int p = 1 - r;

glBegin(GL_POINTS);

// Plot first set of points
plotCirclePoints(xc, yc, x, y);

while(x < y)
{
    x++;

    if(p < 0)
    {
        p = p + 2*x + 1;
    }
    else
    {
        y--;
        p = p + 2*x - 2*y + 1;
    }

    plotCirclePoints(xc, yc, x, y);
}

glEnd();
glFlush();
}

// OpenGL initialization
void init(void)
{
    glClearColor(0.0,0.0,0.0,0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(0.0,0.5,0.0,0.5,-1.0,1.0);
}

// Main function
int main(int argc, char** argv)
{
    printf("Enter Center (x, y): ");
    scanf("%d %d",&xc,&yc);

    printf("Enter Radius: ");
    scanf("%d",&r);

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600,600);
    glutInitWindowPosition(100,100);
    glutCreateWindow("Midpoint Circle Drawing Algorithm");

    init();
    glutDisplayFunc(display);
    glutMainLoop();
}

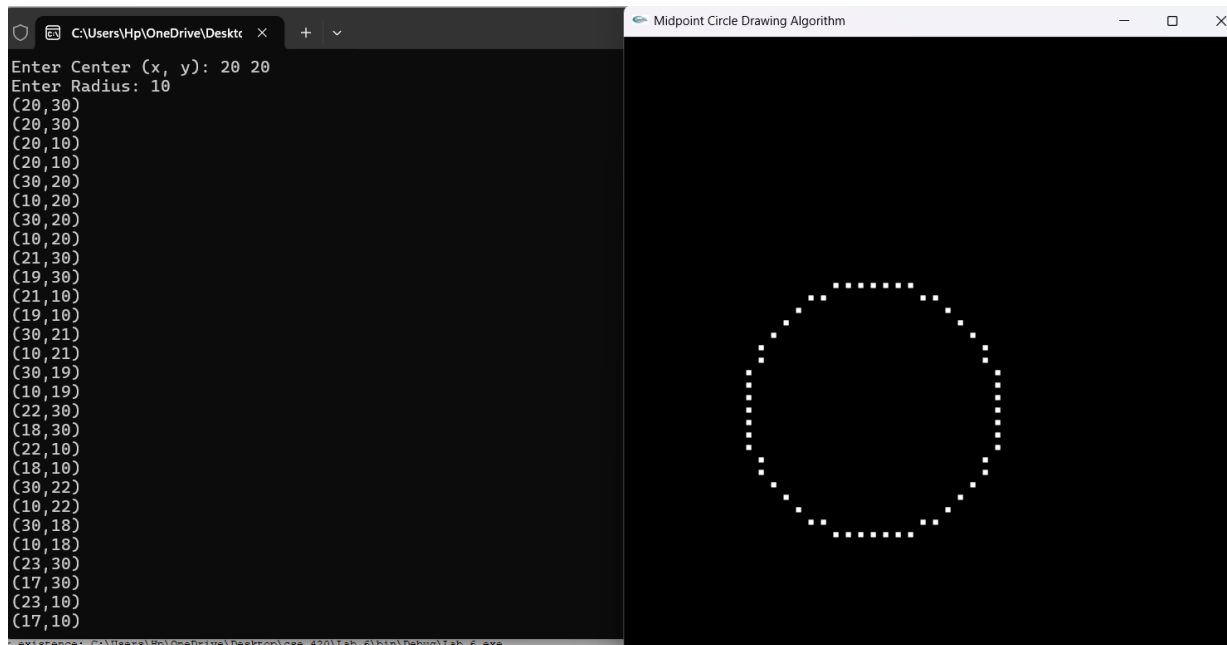
```

```

    return 0;
}

```

Output of Lab Task 1



2. Bresenham's Circle Drawing Algorithm

```

#include<stdio.h>
#include <GL/gl.h>
#include <GL/glut.h>

// Global variables
int xc, yc, r;

// Function to plot the 8 symmetric points
void plotCirclePoints(int xc, int yc, int x, int y)
{
    glVertex2f((xc + x)/100.0, (yc + y)/100.0);
    glVertex2f((xc - x)/100.0, (yc + y)/100.0);
    glVertex2f((xc + x)/100.0, (yc - y)/100.0);
    glVertex2f((xc - x)/100.0, (yc - y)/100.0);

    glVertex2f((xc + y)/100.0, (yc + x)/100.0);
    glVertex2f((xc - y)/100.0, (yc + x)/100.0);
    glVertex2f((xc + y)/100.0, (yc - x)/100.0);
    glVertex2f((xc - y)/100.0, (yc - x)/100.0);

    // Print coordinates in console
}

```

```

    printf("(%d,%d)\n", xc+x, yc+y);
    printf("(%d,%d)\n", xc-x, yc+y);
    printf("(%d,%d)\n", xc+x, yc-y);
    printf("(%d,%d)\n", xc-x, yc-y);
    printf("(%d,%d)\n", xc+y, yc+x);
    printf("(%d,%d)\n", xc-y, yc+x);
    printf("(%d,%d)\n", xc+y, yc-x);
    printf("(%d,%d)\n", xc-y, yc-x);
}

// Display function
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0,1.0,1.0);
    glPointSize(5.0);

    int x = 0;
    int y = r;

    // Bresenham initial decision parameter
    int d = 3 - 2*r;

    glBegin(GL_POINTS);

    plotCirclePoints(xc, yc, x, y);

    while(x < y)
    {
        x++;

        if(d < 0)
        {
            d = d + 4*x + 6;
        }
        else
        {
            y--;
            d = d + 4*(x - y) + 10;
        }

        plotCirclePoints(xc, yc, x, y);
    }

    glEnd();
    glFlush();
}

// OpenGL initialization
void init(void)
{
    glClearColor(0.0,0.0,0.0,0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(0.0,0.5,0.0,0.5,-1.0,1.0);
}

```

```
// Main function
int main(int argc, char** argv)
{
    printf("Enter Center (x, y): ");
    scanf("%d %d", &xc, &yc);

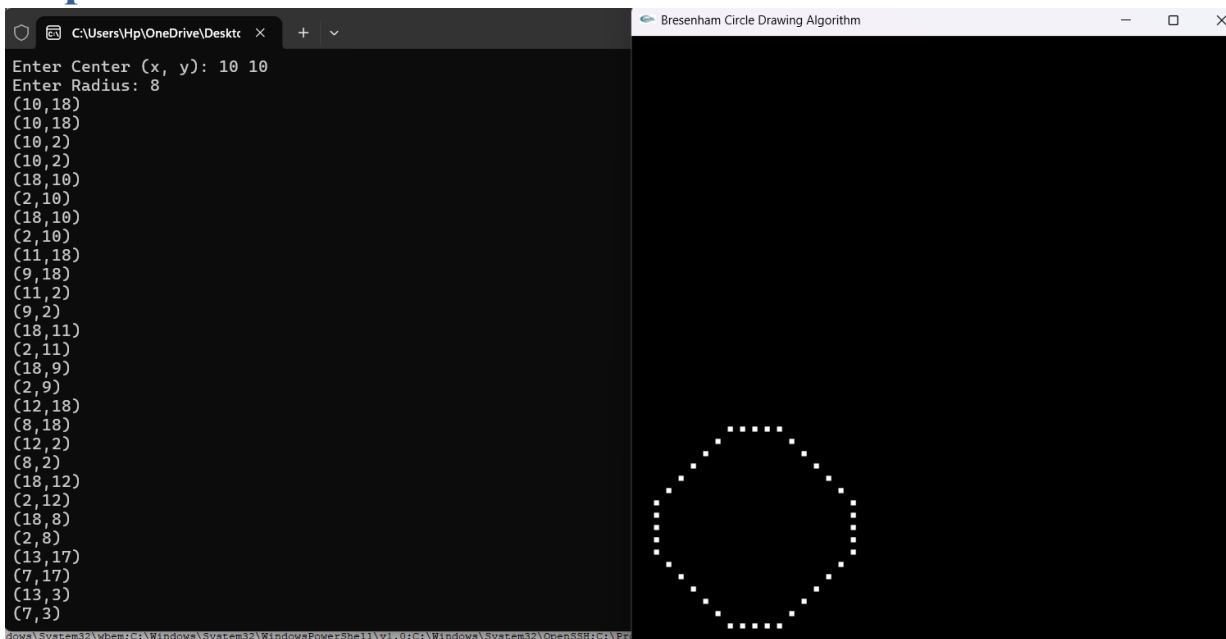
    printf("Enter Radius: ");
    scanf("%d", &r);

    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 600);
    glutInitWindowPosition(100, 100);
    glutCreateWindow("Bresenham Circle Drawing Algorithm");

    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}
```

Output of Lab Task 2



Comparison between Midpoint Circle Algorithm and Bresenham Circle Algorithm

Feature	Midpoint Circle Algorithm	Bresenham Circle Algorithm
Initial Decision Parameter	$p = 1 - r$	$d = 3 - 2r$
Update if parameter < 0	$p = p + 2x + 1$	$d = d + 4x + 6$
Update if parameter ≥ 0	$p = p + 2x - 2y + 1$	$d = d + 4(x - y) + 10$
Pixel Decision	Midpoint between two candidate pixels	Uses integer incremental error
Arithmetic Complexity	Slightly simpler formulas	Slightly more optimized for integer arithmetic
Operations Used	Addition and subtraction	Only integer addition
Symmetry Used	8-way symmetry	8-way symmetry
Accuracy	Very good	Very good
Speed	Fast	Slightly faster in hardware implementations
Implementation Complexity	Easier to understand	Slightly more complex formulas

Midpoint Circle Drawing Algorithm

Advantages

- **Easy to understand and implement.**
- **Uses simple arithmetic operations.**
- **Produces accurate circle shapes.**
- **Uses symmetry to reduce calculations.**

Disadvantages

- **Slightly slower than Bresenham algorithm.**
- **Still limited to pixel-based (raster) graphics systems.**

Bresenham's Circle Drawing Algorithm

Advantages

- Uses only **integer calculations**.
- **Very fast and efficient**.
- Requires **less computation**.
- Suitable for **raster graphics displays**.

Disadvantages

- **More difficult to understand** than midpoint algorithm.
- The **decision parameter formula is more complex**.